Model Question Paper – Cloud Data Engineer - Jain University

HCLTECH ASSESSMENT – 2 REPORT

Name: Juved MahammadRafik Attar

Roll number: 2JI23CS056

College: Jain College of Engineering, Belagavi, Karnataka

Programme: B.E. — Computer Science and Engineering

Batch: 2023–2027

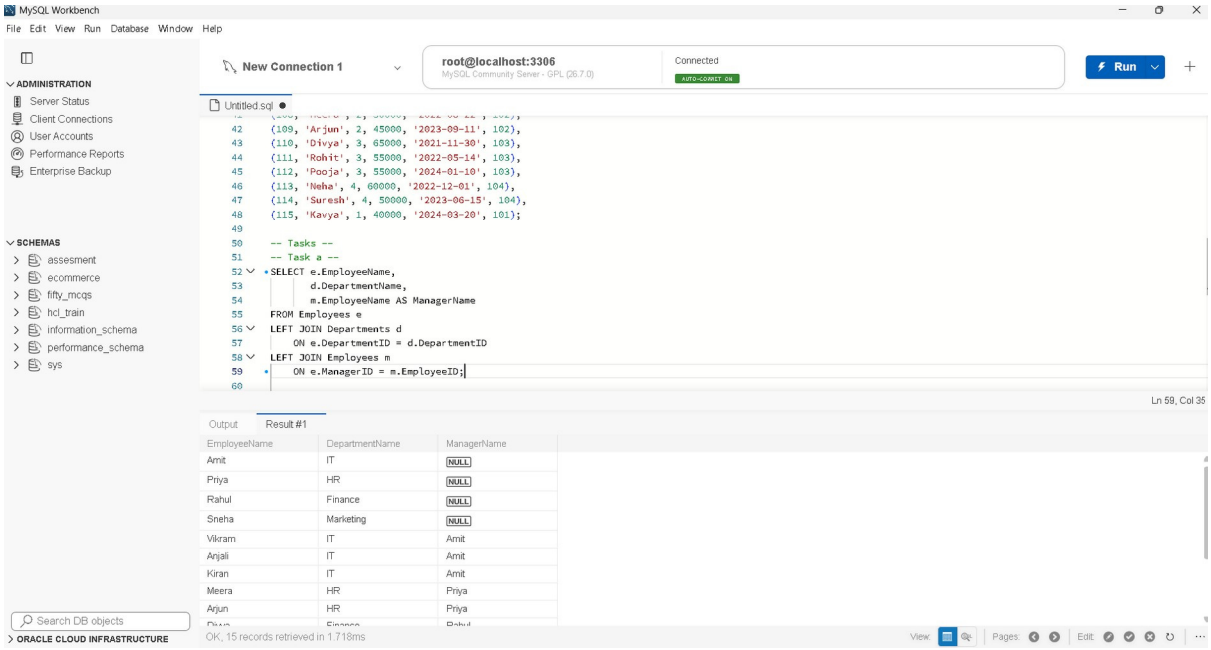
Submission date: 11 October 2026

SECTION A: ADVANCED SQL | Maximum Marks: 20

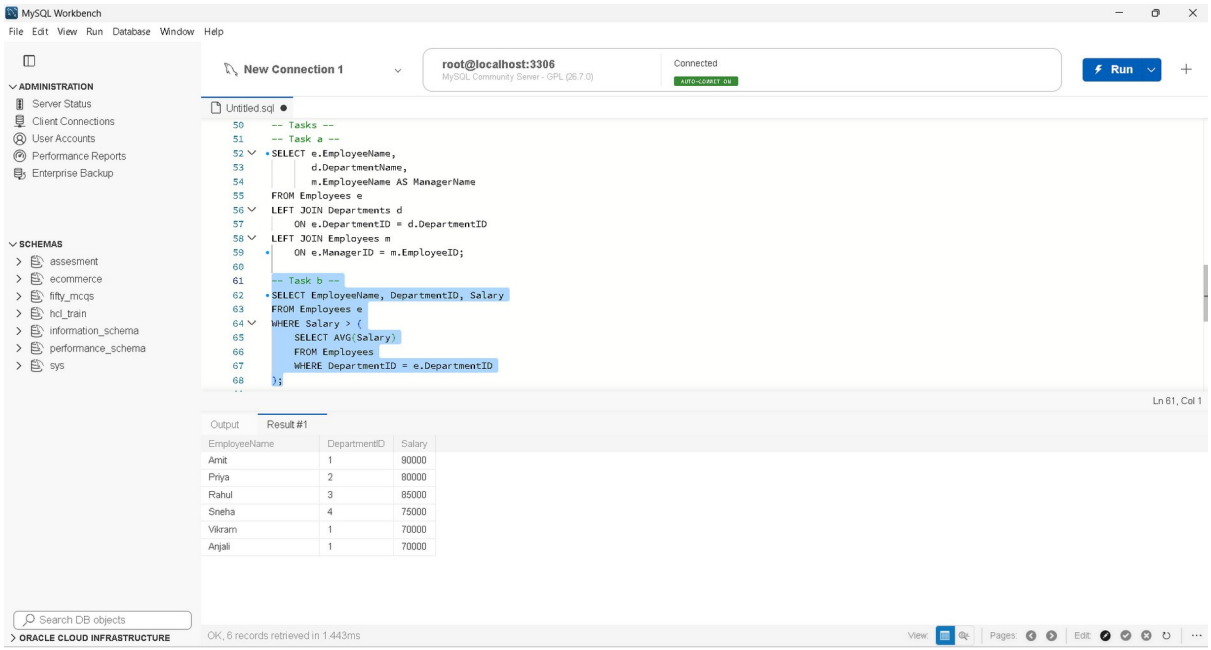
Question 1: Database Design and Advanced Queries (8 Marks)

Tasks :

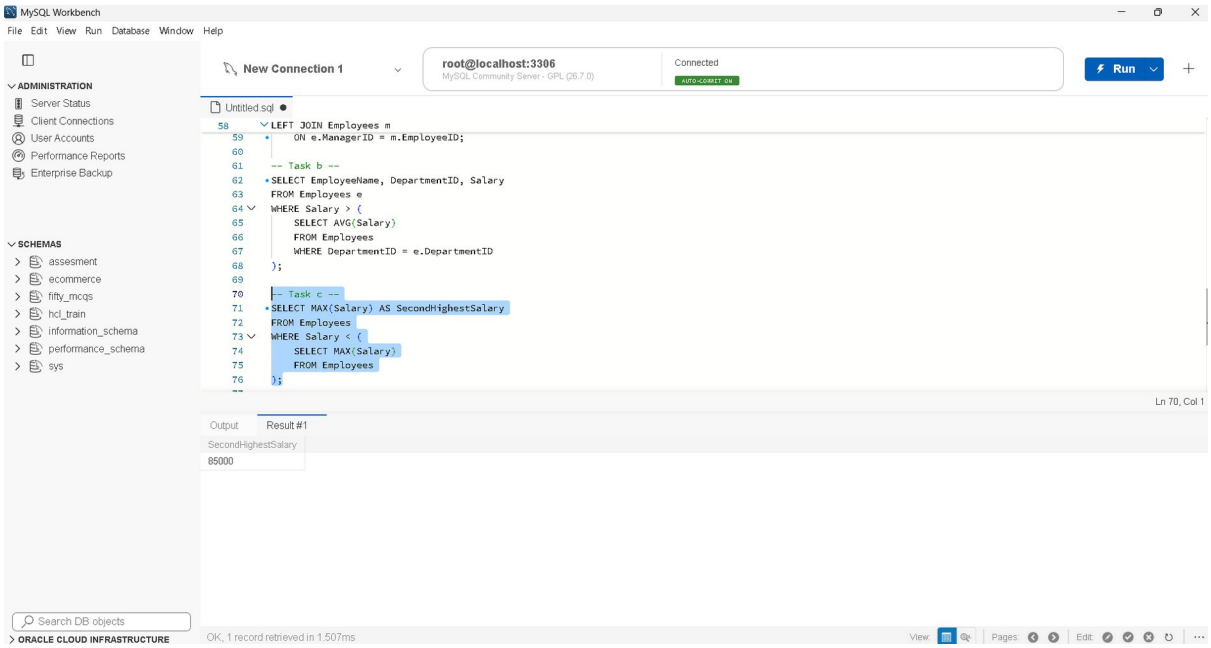
a. Display employee names, department names, and manager names using joins. (2 marks)



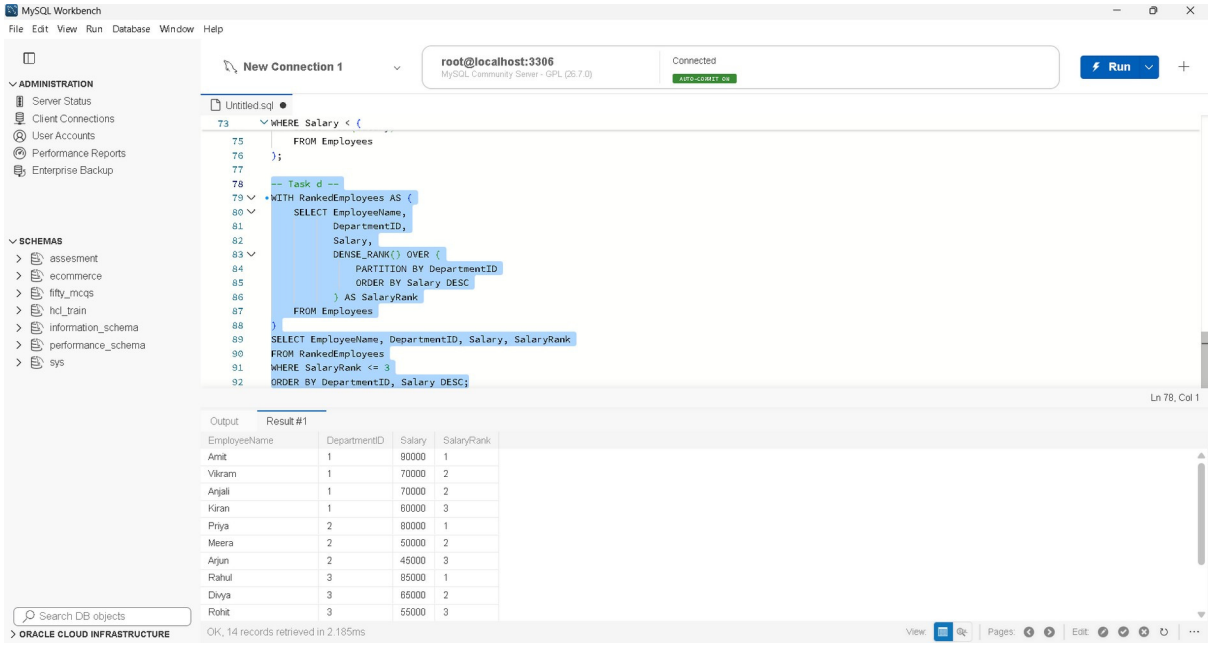
b. Find employees earning more than their department's average salary. (2 marks)



c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. (2 marks)



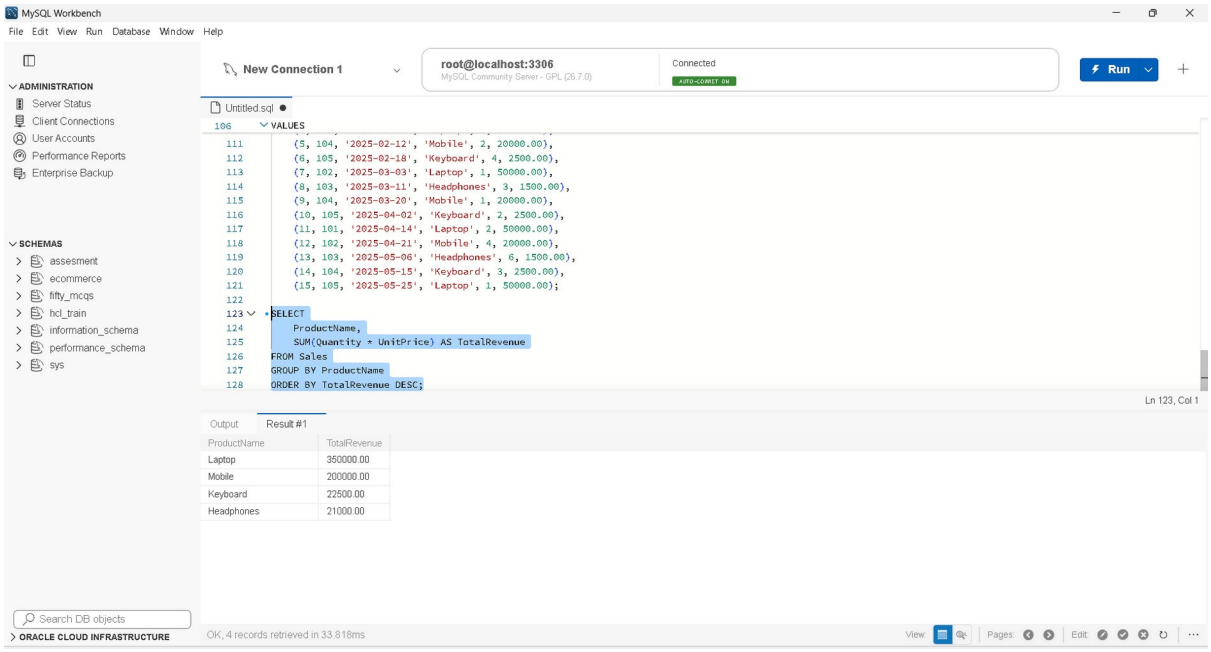
d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff. (2 marks)



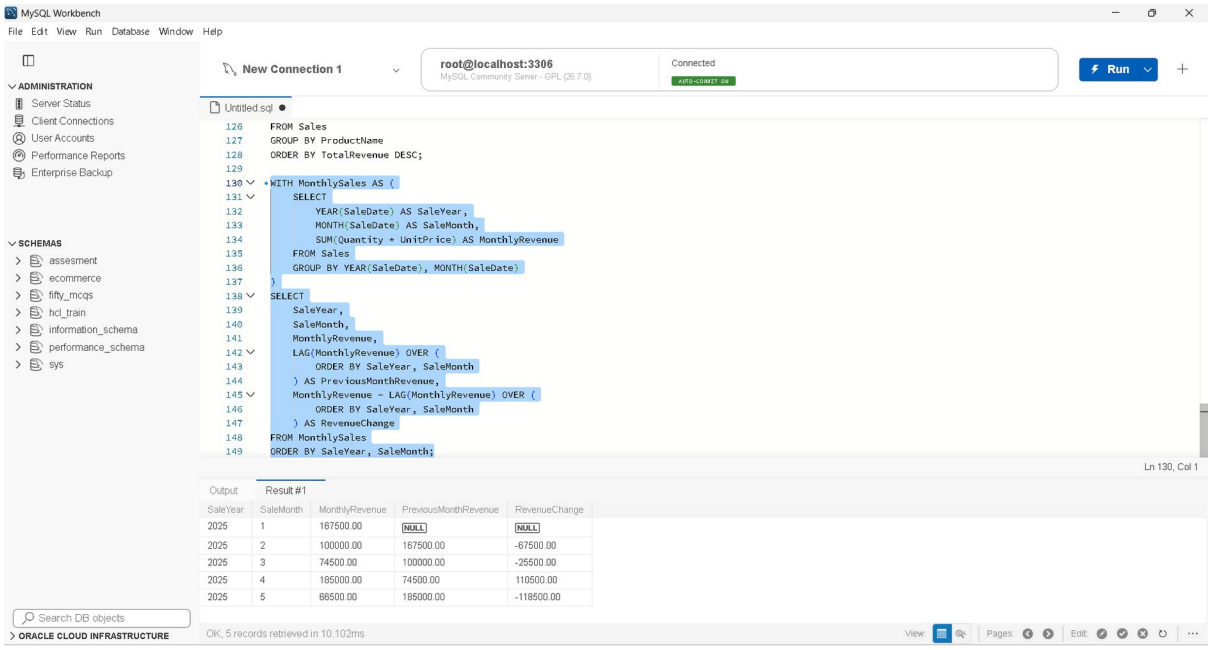
Question 2: Sales Analytics and Query Optimization (6 Marks)

Tasks :

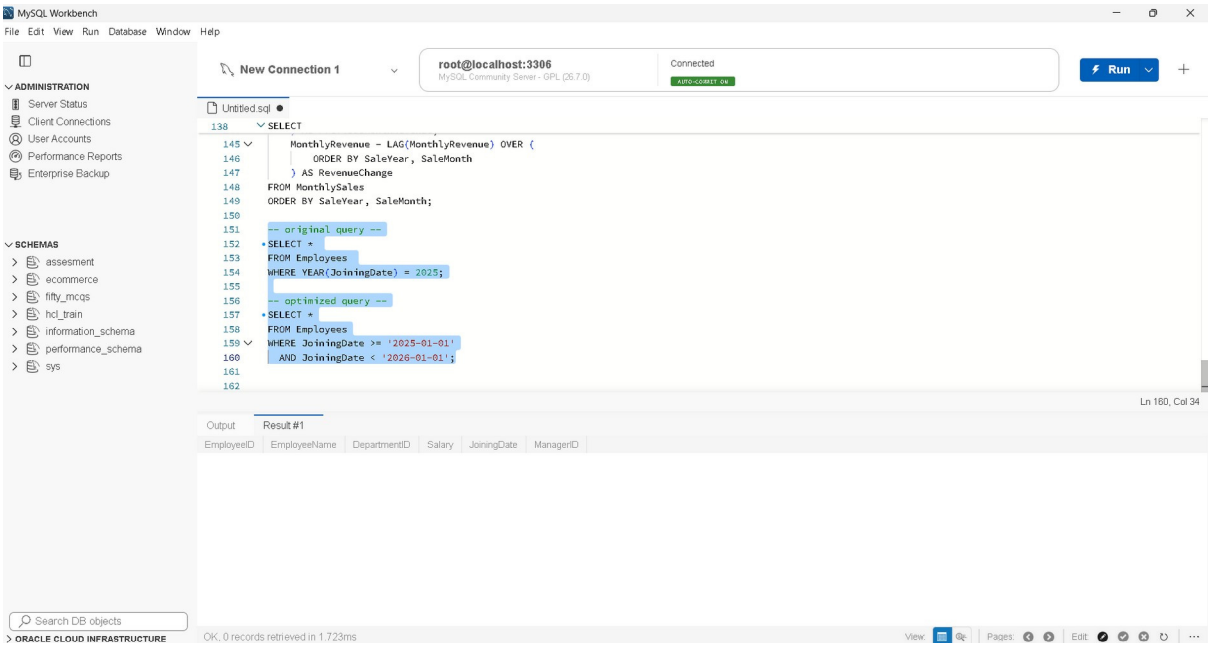
a. Calculate total revenue for each product. (2 marks)



b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)



C. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks) SELECT * FROM Employees WHERE YEAR(JoiningDate) = 2025; Assume JoiningDate may be indexed.



Question 3: SQL Troubleshooting (6 Marks)

Tasks :

a. Explain how an execution plan helps identify expensive operations in a query. (2 marks)

Ans : An execution plan shows the steps the database uses to execute a query.

- It highlights costly operations such as full table scans, expensive joins, and sorts.
- It helps identify performance bottlenecks so that indexes or queries can be optimized

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. (2 marks)

Ans : Create a composite index on DepartmentID and Salary:

```
sql

CREATE INDEX idx_employee_dept_salary
ON Employees (DepartmentID, Salary);
```

Reasoning:

- The index supports queries filtering by both DepartmentID and Salary.
- Placing DepartmentID first is suitable when queries filter by department alone or by department and salary together.

c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once. (2 marks)

Ans :

```
SELECT EmailID
FROM Employees
GROUP BY EmailID
HAVING COUNT(EmailID) > 1;
```

SECTION B: LINUX | Maximum Marks: 15

Question 4 : Log Analysis

Tasks :

a. Find all lines containing ERROR. (1 mark)

Ans : grep "ERROR" /var/log/application.log

b. Display the 20 most recent error entries in the file. (1 mark)

Ans : grep "ERROR" /var/log/application.log | tail -n 20

c. Count the total number of error entries. (1 mark)

Ans : grep -c "ERROR" /var/log/application.log

d. Extract error entries for a specified date into a separate file. (1 mark)

Ans : grep "^2026-10-28.*ERROR" /var/log/application.log > errors_2026-10-28.log

e. Explain how you would calculate the number of errors occurring in each hour. (1 mark)

Ans : grep “^2026-10-28.last_hour.ERROR” /var/log/application.log > errors_last_hour.log

Question 5: Permissions and System Troubleshooting (5 Marks)

Tasks :

a. Create a group named finance and a report file, assuming you have the required privileges. (1 mark)

Ans : sudo groupadd finance

```
touch report.txt
```

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. (1 mark)

Ans : sudo chown "$USER":finance report.txt

```
    chmod 640 report.txt
```

c. Identify the processes consuming the most CPU. (1 mark)

Ans : top

d. Display memory usage and filesystem disk utilization. (1 mark)

Ans : free -h

```
df -h
```

e. Find files larger than 100 MB under a specified application directory. (1 mark)

Ans : find /path/to/application -type f -size +100M

Question 6: Bash Scripting (5 Marks)

Write a Bash script named backup_check.sh :

SCRIPT :

```
#!/bin/bash
# Check whether both arguments are provided

if [ "$#" -ne 2 ]; then
    echo "Error: Usage: $0 <source_directory> <destination_directory>"

    exit 1
fi
SOURCE="$1"
DESTINATION="$2"
# Check whether the source directory exists

if [ ! -d "$SOURCE" ]; then
    echo "Error: Source directory '$SOURCE' does not exist."

    exit 1
fi
# Create destination directory if it does not exist

if [ ! -d "$DESTINATION" ]; then
    mkdir -p "$DESTINATION"

  if [ $? -ne 0 ]; then
      echo "Error: Could not create destination directory."

      exit 1
  fi
fi
# Generate a timestamp

TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
# Create a compressed archive

ARCHIVE="$DESTINATION/backup_$TIMESTAMP.tar.gz"
if tar -czf "$ARCHIVE" -C "$(dirname "$SOURCE")" "$(basename "$SOURCE")"; then

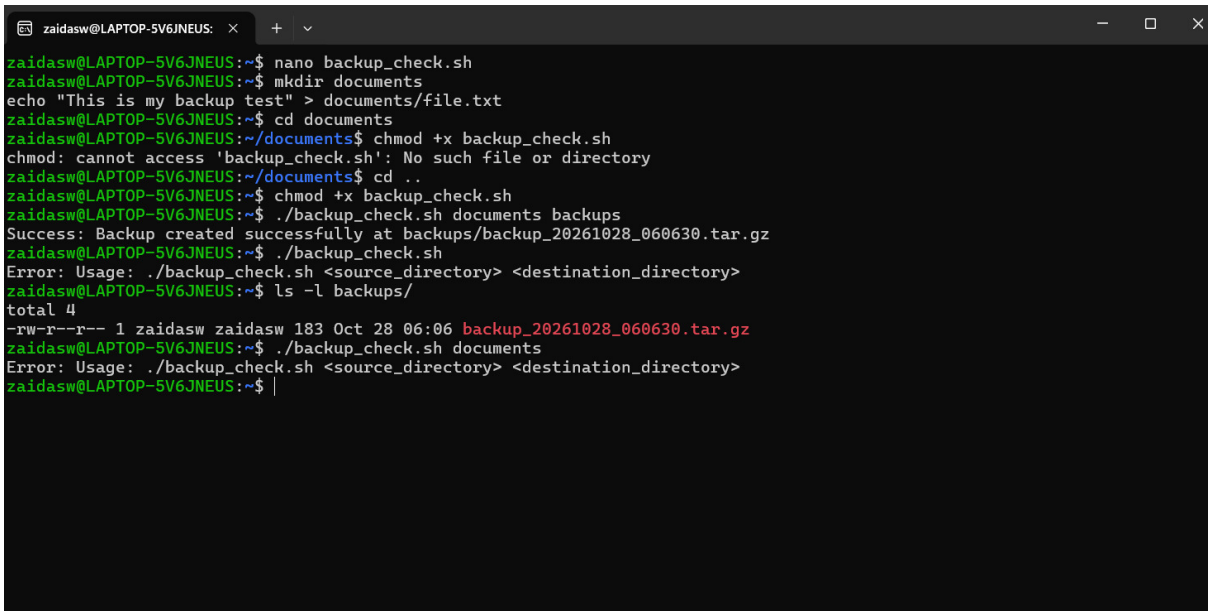
    echo "Success: Backup created successfully at $ARCHIVE"

else

    echo "Error: Backup failed."

    exit 1
fi
exit 0
```

Execution :


zaidasw@LAPTOP-5V6JNEUS:
zaidasw@LAPTOP-5V6JNEUS:~$ nano backup_check.sh
zaidasw@LAPTOP-5V6JNEUS:~$ mkdir documents
echo "This is my backup test" > documents/file.txt
zaidasw@LAPTOP-5V6JNEUS:~$ cd documents
zaidasw@LAPTOP-5V6JNEUS:~/documents$ chmod +x backup_check.sh
chmod: cannot access 'backup_check.sh': No such file or directory
zaidasw@LAPTOP-5V6JNEUS:~/documents$ cd ..
zaidasw@LAPTOP-5V6JNEUS:~$ chmod +x backup_check.sh
zaidasw@LAPTOP-5V6JNEUS:~$ ./backup_check.sh documents backups
Success: Backup created successfully at backups/backup_20261028_060630.tar.gz
zaidasw@LAPTOP-5V6JNEUS:~$ ./backup_check.sh
Error: Usage: ./backup_check.sh <source_directory> <destination_directory>
zaidasw@LAPTOP-5V6JNEUS:~$ ls -l backups/
total 4
-rw-r--r-- 1 zaidasw zaidasw 183 Oct 28 06:06 backup_20261028_060630.tar.gz
zaidasw@LAPTOP-5V6JNEUS:~$ ./backup_check.sh documents
Error: Usage: ./backup_check.sh <source_directory> <destination_directory>
zaidasw@LAPTOP-5V6JNEUS:~$

SECTION C: PYSPARK DATA ENGINEERING | Maximum Marks: 20

Question 7: PySpark DataFrame Operations (6 Marks)

[1]:
```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

spark = SparkSession.builder \
    .appName("Question7") \
    .master("local[*]") \
    .getOrCreate()

data = [
    (1, "Amit", "IT", 75000, "2022-01-10"),
    (2, "Priya", "HR", 45000, "2021-03-15"),
    (3, "Rahul", "IT", 85000, "2020-06-20"),
    (4, "Sneha", "Finance", 65000, "2023-02-01"),
    (5, "Kiran", "HR", 55000, "2022-08-12"),
    (6, "Neha", "IT", 92000, "2019-11-05"),
    (7, "Arjun", "Finance", 70000, "2021-07-18"),
    (8, "Divya", "IT", 60000, "2023-04-10")
]

columns = [
    "EmployeeID", "EmployeeName", "Department",
    "Salary", "JoiningDate"
]

df = spark.createDataFrame(data, columns)
df = df.withColumn("JoiningDate", col("JoiningDate").cast("date"))

print("a. Schema:")
df.printSchema()

print("First five records:")
df.show(5, truncate=False)

print("b. Employees earning more than 60000:")
df.filter(col("Salary") > 60000).show()

print("c. Annual Salary:")
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.select("EmployeeName", "Salary", "AnnualSalary").show()

print("d. Department-wise average salary and employee count:")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()

print("e. Five highest-paid employees:")
df.orderBy(col("Salary").desc()).show(5)
```

```
C:\Users\Anmol\dataEngineer\.venv\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0.
Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
a. Schema:
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

First five records:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|1         |Amit        |IT        |75000 |2022-01-10 |
|2         |Priya       |HR        |45000 |2021-03-15 |
|3         |Rahul       |IT        |85000 |2020-06-20 |
|4         |Sneha       |Finance   |65000 |2023-02-01 |
|5         |Kiran       |HR        |55000 |2022-08-12 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
b. Employees earning more than 60000:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Amit|        IT| 75000| 2022-01-10|
|         3|       Rahul|        IT| 85000| 2020-06-20|
|         4|       Sneha|   Finance| 65000| 2023-02-01|
|         6|        Neha|        IT| 92000| 2019-11-05|
|         7|       Arjun|   Finance| 70000| 2021-07-18|
+----------+------------+----------+------+-----------+

c. Annual Salary:
+------------+------+------------+
|EmployeeName|Salary|AnnualSalary|
+------------+------+------------+
|        Amit| 75000|      900000|
|       Priya| 45000|      540000|
|       Rahul| 85000|     1020000|
|       Sneha| 65000|      780000|
|       Kiran| 55000|      660000|
|        Neha| 92000|     1104000|
|       Arjun| 70000|      840000|
|       Divya| 60000|      720000|
+------------+------+------------+

d. Department-wise average salary and employee count:
+----------+-------------+-------------+
|Department|AverageSalary|EmployeeCount|
+----------+-------------+-------------+
|        IT|      78000.0|            4|
|        HR|      50000.0|            2|
|   Finance|      67500.0|            2|
+----------+-------------+-------------+

e. Five highest-paid employees:
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         6|        Neha|        IT| 92000| 2019-11-05|     1104000|
|         3|       Rahul|        IT| 85000| 2020-06-20|     1020000|
|         1|        Amit|        IT| 75000| 2022-01-10|      900000|
|         7|       Arjun|   Finance| 70000| 2021-07-18|      840000|
|         4|       Sneha|   Finance| 65000| 2023-02-01|      780000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

Question 8: Data Cleaning and Aggregation (7 Marks)

```
[4]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date, sum
from pyspark.sql.functions import expr


spark = SparkSession.builder.appName("DataCleaning").getOrCreate()

df = spark.read.csv("transactions.csv", header=True, inferSchema=True)

print("Original Data:")
df.show()

# Remove duplicates
df = df.dropDuplicates(["TransactionID"])


df = df.withColumn(
    "TransactionDate",
    expr("try_to_date(TransactionDate, 'yyyy-MM-dd')")
)
```

```
df = df.filter(
    col("CustomerID").isNotNull() &
    col("TransactionDate").isNotNull() &
    col("Quantity").isNotNull() &
    col("UnitPrice").isNotNull()
)

df = df.withColumn("Revenue", col("Quantity") * col("UnitPrice"))

print("Cleaned Data:")
df.show()

print("Revenue by Product:")
df.groupBy("Product").agg(
    sum("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc()).show()
```

```
Original Data:
+-------------------+----------+---------------+--------+--------+---------+
|      TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------------+----------+---------------+--------+--------+---------+
|2026-10-28 01:00:00|        C1|     2026-01-10|  Laptop|       1|    50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 03:00:00|        C3|        invalid|Keyboard|       1|     1500|
|2026-10-28 04:00:00|      NULL|     2026-01-12|  Laptop|       1|    50000|
+-------------------+----------+---------------+--------+--------+---------+

Cleaned Data:
+-------------------+----------+---------------+-------+--------+---------+-------+
|      TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue|
+-------------------+----------+---------------+-------+--------+---------+-------+
|2026-10-28 01:00:00|        C1|     2026-01-10| Laptop|       1|    50000|  50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|  Mouse|       2|      500|   1000|
+-------------------+----------+---------------+-------+--------+---------+-------+

Revenue by Product:
+-------+------------+
|Product|TotalRevenue|
+-------+------------+
| Laptop|       50000|
|  Mouse|        1000|
+-------+------------+
```

Question 9: PySpark Joins and Performance (7 Marks)

```
[1]: from pyspark.sql import SparkSession
     from pyspark.sql.functions import col, sum, broadcast

     spark = SparkSession.builder \
         .appName("CustomerOrders") \
         .getOrCreate()

     customers_data = [
         (1, "Amit", "Bengaluru"),
         (2, "Priya", "Mumbai"),
         (3, "Rahul", "Delhi"),
         (4, "Sneha", "Bengaluru"),
         (5, "Kiran", "Chennai")
     ]

     orders_data = [
         (101, 1, 5000.0, "2026-01-10"),
         (102, 2, 3000.0, "2026-01-12"),
         (103, 1, 2000.0, "2026-01-15"),
         (104, 3, 7000.0, "2026-01-18"),
         (105, 4, 4000.0, "2026-01-20")
     ]

     customers = spark.createDataFrame(
         customers_data,
         ["CustomerID", "CustomerName", "City"]
     )

     orders = spark.createDataFrame(
         orders_data,
         ["OrderID", "CustomerID", "OrderAmount", "OrderDate"]
     )

     print("Customers:")
     customers.show()

     print("Orders:")
     orders.show()
```

```
C:\Users\Anmol\dataEngineer\.venv\Lib\site-packages\pyspark\testing
0. Some features may not work correctly. It is recommended to use pa
  require_minimum_pandas_version()
Customers:
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         1|        Amit|Bengaluru|
|         2|       Priya|   Mumbai|
|         3|       Rahul|    Delhi|
|         4|       Sneha|Bengaluru|
|         5|       Kiran|  Chennai|
+----------+------------+---------+

Orders:
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    101|         1|     5000.0|2026-01-10|
|    102|         2|     3000.0|2026-01-12|
|    103|         1|     2000.0|2026-01-15|
|    104|         3|     7000.0|2026-01-18|
|    105|         4|     4000.0|2026-01-20|
+-------+----------+-----------+----------+
```

```
joined = orders.join(customers, "CustomerID", "inner")

print("Orders with customer details:")
joined.show()
```

```
Orders with customer details:
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    101|     5000.0|2026-01-10|        Amit|Bengaluru|
|         1|    103|     2000.0|2026-01-15|        Amit|Bengaluru|
|         2|    102|     3000.0|2026-01-12|       Priya|   Mumbai|
|         3|    104|     7000.0|2026-01-18|       Rahul|    Delhi|
|         4|    105|     4000.0|2026-01-20|       Sneha|Bengaluru|
+----------+-------+-----------+----------+------------+---------+
```

```
[3]: print("Customers without orders:")

customers.join(
    orders.select("CustomerID").distinct(),
    "CustomerID",
    "left_anti"
).show()
```

```
Customers without orders:
+----------+------------+-------+
|CustomerID|CustomerName|   City|
+----------+------------+-------+
|         5|       Kiran|Chennai|
+----------+------------+-------+
```

```
print("Broadcast join:")

broadcast(orders).join(customers, "CustomerID").show()
```

```
Broadcast join:
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    103|     2000.0|2026-01-15|        Amit|Bengaluru|
|         1|    101|     5000.0|2026-01-10|        Amit|Bengaluru|
|         2|    102|     3000.0|2026-01-12|       Priya|   Mumbai|
|         3|    104|     7000.0|2026-01-18|       Rahul|    Delhi|
|         4|    105|     4000.0|2026-01-20|       Sneha|Bengaluru|
+----------+-------+-----------+----------+------------+---------+
```

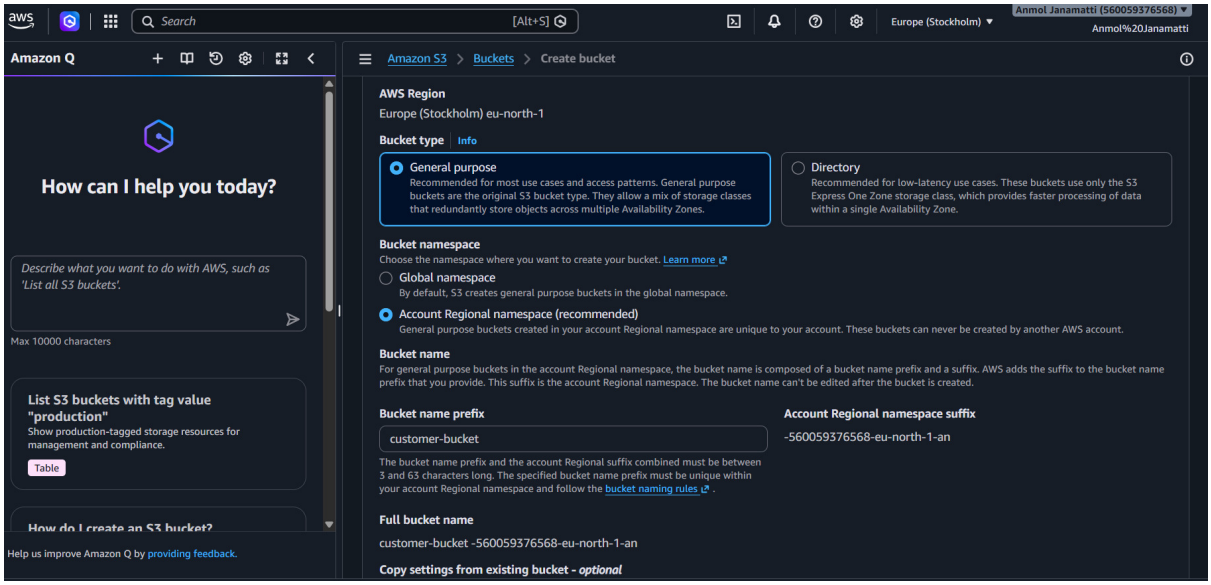
f. Transformations vs. Actions (1 Mark)

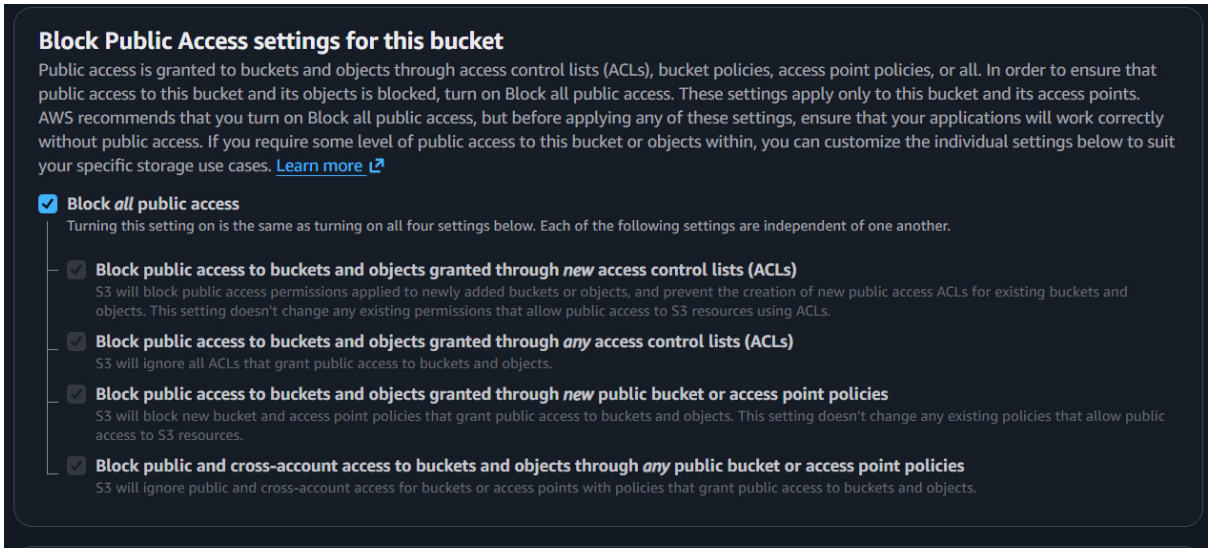
- **Transformations:** Create a new DataFrame or execution plan. Example: filter().
- **Actions:** Execute computations and return results. Example: show().

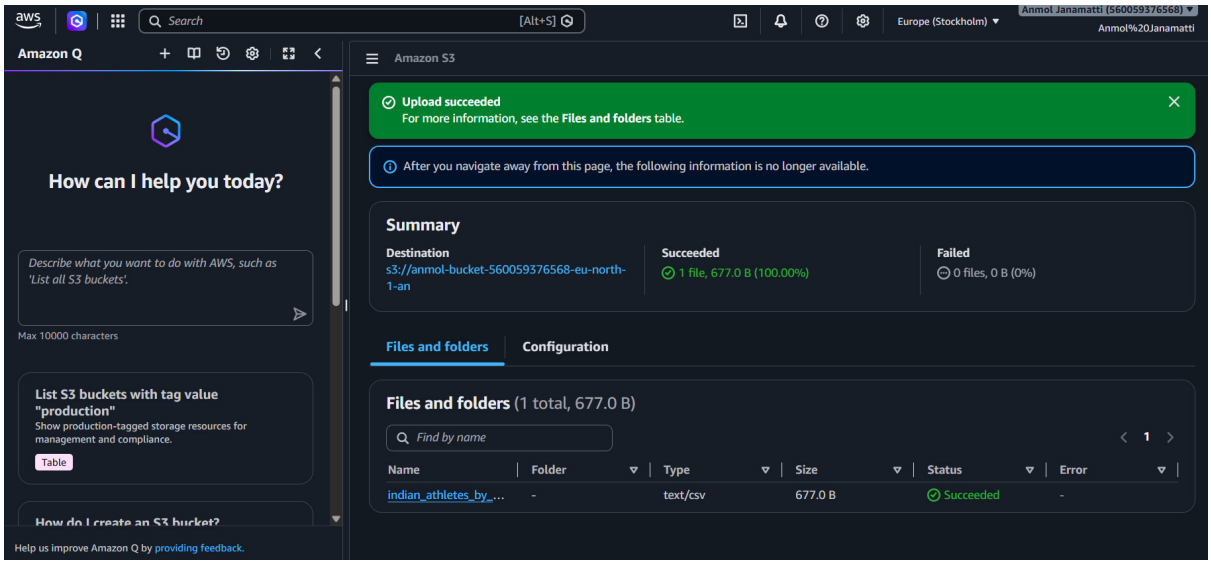
SECTION D: AWS CLOUD SERVICES | Maximum Marks: 15

Question 10: S3 and IAM Security (5 Marks)

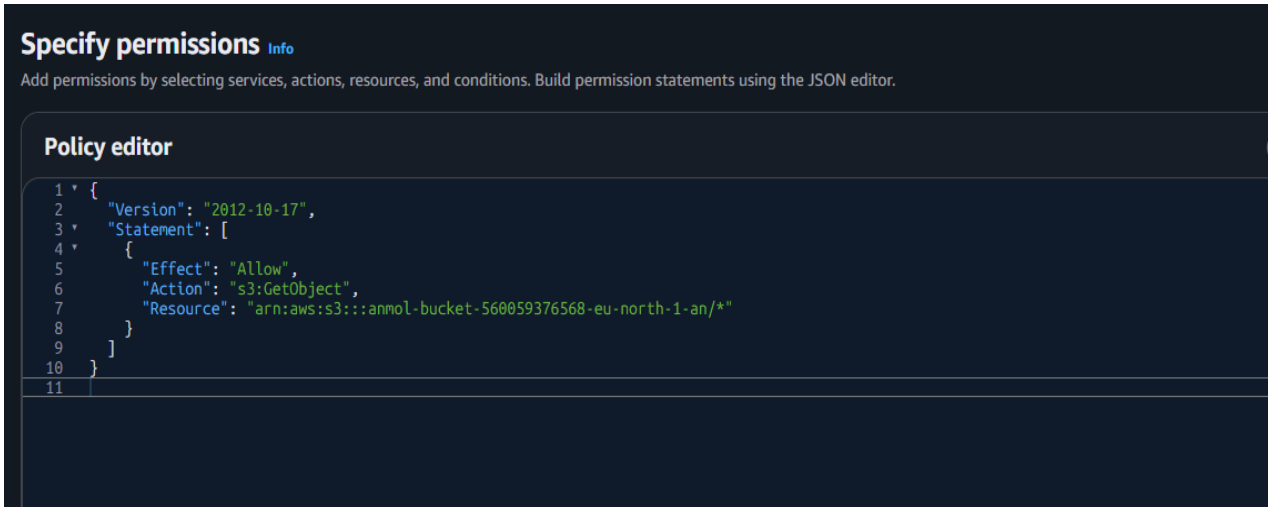
a. Create an S3 bucket with Block Public Access enabled and upload a sample file. (1 mark)







b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)



c. Explain how you would verify that an unauthorized user cannot read the file. (1 mark)

answer: If an unauthorized user tries to access the file, an Access Denied (403) error will occur. We can verify the failed access attempt using AWS CloudTrail logs (after enabling S3 data events).

d. Explain how encryption at rest protects the stored data. (1 mark)

answer: Encryption at rest protects stored data by encrypting it using encryption keys. Even if an unauthorized person gains access to the storage, they cannot read the data without the required decryption key. In AWS S3, this can be implemented using Server-Side Encryption (SSE).

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks)

a. Check the EC2 instance state and status checks.

Open the AWS EC2 console and check whether the instance is in the Running state. Verify that both system status checks and instance status checks are passing.

b. Explain how security groups and route tables can affect application connectivity.

Security groups control incoming and outgoing traffic to the EC2 instance. Incorrect rules may block application traffic. Route tables determine where network traffic is sent, and incorrect routes can prevent users from connecting to the application.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

Open Amazon CloudWatch and create an alarm for the EC2 instance's CPUUtilization metric. Set the threshold to greater than 85% and define the evaluation period. Configure an SNS notification to alert the administrator when the alarm is triggered.

d. Identify one method for investigating the root cause of high CPU utilization.

Use the EC2 instance's operating system tools, such as the top command in Linux, to identify processes consuming high CPU resources.

Question 12: Event-Driven Processing (5 Marks)

A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location. Design a solution using AWS services.

a. Identify a suitable service for executing the processing code. (1 mark)

AWS Lambda is suitable for executing code to validate and process uploaded CSV files automatically.

b. Explain how an S3 upload can trigger processing. (1 mark)

Configure an S3 event notification to trigger the Lambda function whenever a new CSV file is uploaded to the input bucket.

c. Explain the IAM permissions needed to read the input file and write the results. (1 mark)

Assign an IAM execution role to Lambda with s3:GetObject permission for the input bucket and s3:PutObject permission for the output bucket.

d. Explain how errors and processing logs can be captured. (1 mark)

Use Amazon CloudWatch Logs to store Lambda execution logs and errors. CloudWatch can also be used to monitor failures.

e. Explain how to prevent duplicate processing or recursive triggering.

Use a separate S3 bucket for the processed results so that output uploads do not trigger the same Lambda function. To prevent duplicate processing, make the function idempotent by tracking processed files.

SECTION E: AWS STEP FUNCTIONS | Maximum Marks: 15

Question 13: State Machine Design (7 Marks)

a. Identify suitable states for validation, processing, storage, and notification. (2 marks)

Use AWS Step Functions with the following states: Task states for validation, data transformation, sales calculation, S3 storage, and sending the completion notification. Use Choice to check whether the input file is valid and Fail to handle invalid files.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. (1 mark)

The Choice state checks the validation result. If the file is valid, it moves to processing. If the file is invalid, it routes the workflow to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors. (2 marks)

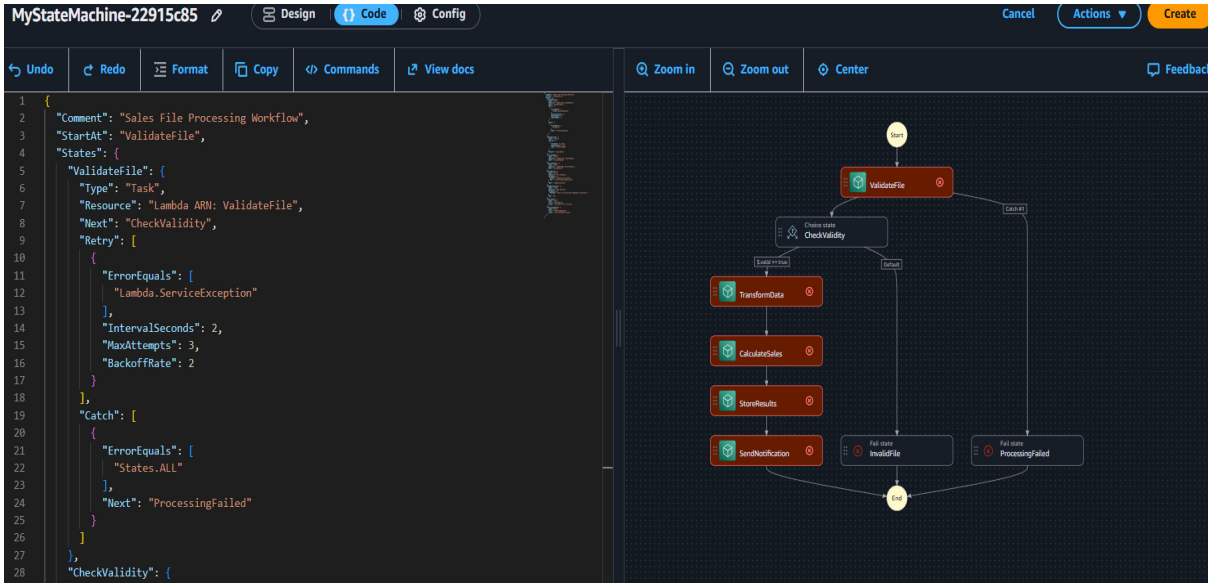
Retry automatically retries temporary failures, such as service errors. Catch handles errors that cannot be recovered by retrying and routes the workflow to an error-handling state or Fail state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services. (1 mark)

Step Functions can invoke AWS Lambda functions using Task states to validate, transform, and calculate sales. It can also use supported AWS service integrations to store results in S3 and send notifications through Amazon SNS.

e. Explain how execution history helps troubleshoot a failed workflow. (1 mark)

Step Functions execution history shows each state's input, output, status, and error details. This helps identify the failed state and troubleshoot the problem.



Question 14: Step Functions Implementation (8 Marks)

A data engineering team has developed three processing functions:

• ValidateFile

• TransformData

• LoadResults

The functions must execute in sequence. Requirements:

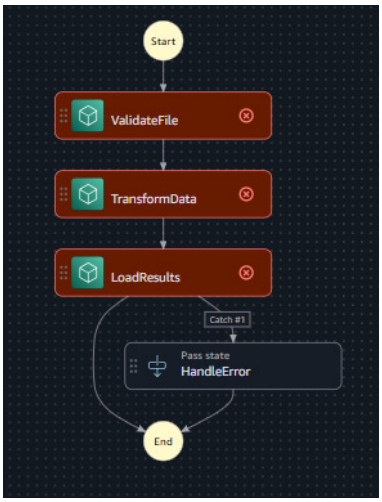
• If validation fails, the workflow must stop.

• If transformation fails temporarily, the workflow must retry.

• If loading fails permanently, the workflow must record the failure. Tasks:

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

```
{
  "Comment": "Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "Lambda ARN: ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "Lambda ARN: TransformData",
      "Retry": [
        {
          "ErrorEquals": [
            "Lambda.ServiceException"
          ],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2
        }
      ],
      "Next": "LoadResults"
    },
    "LoadResults": {
      "Type": "Task",
      "Resource": "Lambda ARN: LoadResults",
      "Catch": [
        {
          "ErrorEquals": [
            "States.ALL"
          ],
          "Next": "HandleError"
        }
      ],
      "End": true
    },
    "HandleError": {
      "Type": "Pass",
      "Result": "Loading failed",
      "End": true
    }
  }
}
```



b. Configure a retry policy for the transformation task. (2 marks)

The Retry state retries temporary errors such as Lambda.ServiceException. It waits for 2 seconds, retries up to 3 times, and doubles the wait time after each retry using BackoffRate: 2.

c. Configure a Catch transition for the loading task. (1 mark)

The Catch block captures loading errors and transfers control to the HandleError state, where the failure is recorded.

d. Explain how data from one task is passed to the next task. (1 mark)

Step Functions passes the output of one task as the input to the next task by default. JSON data can be used to share processing results between functions.

e. Explain how you would test successful and failed executions. (1 mark)

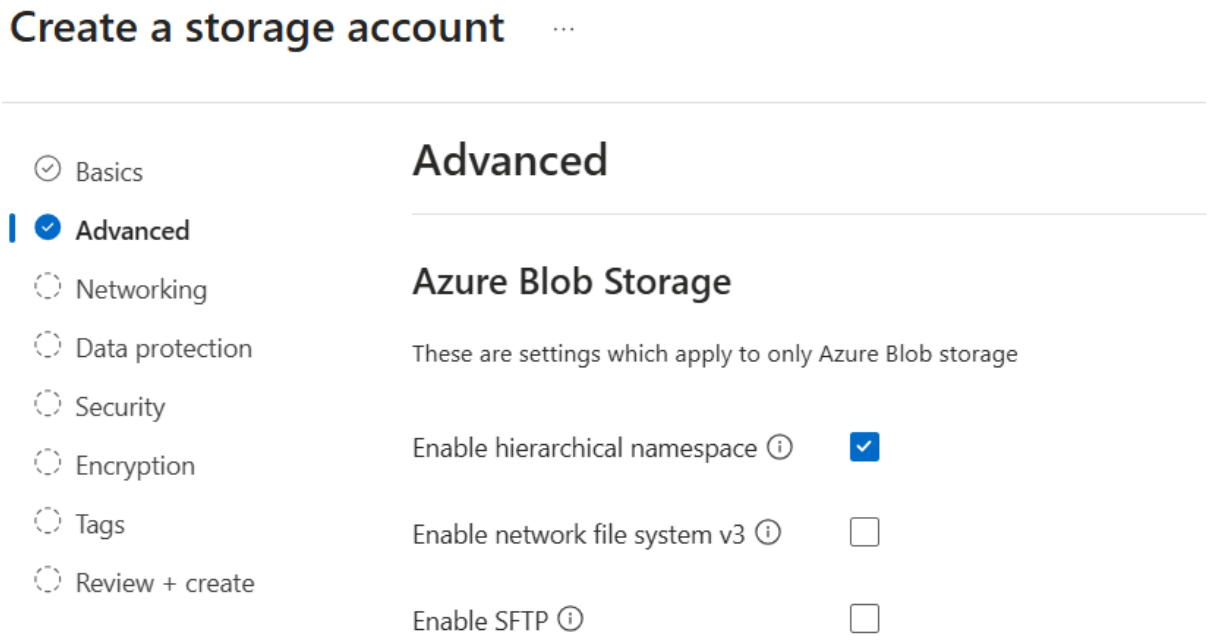
Run the workflow with valid input to verify successful completion. Test with invalid input and simulated transformation or loading errors to verify that retries and error handling work correctly.

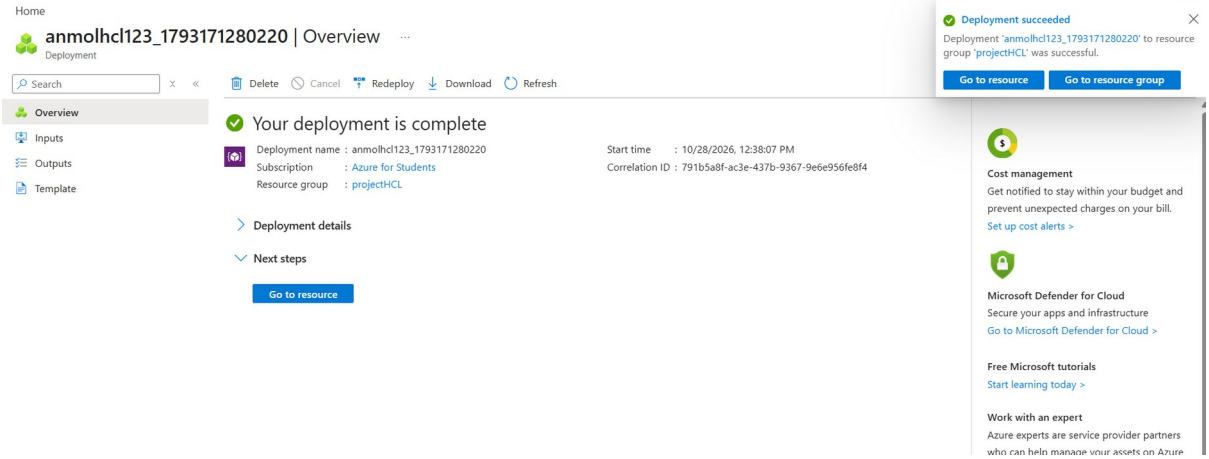
SECTION F: MICROSOFT AZURE | Maximum Marks: 15

Question 15: Azure Data Lake Storage and Security (5 Marks)

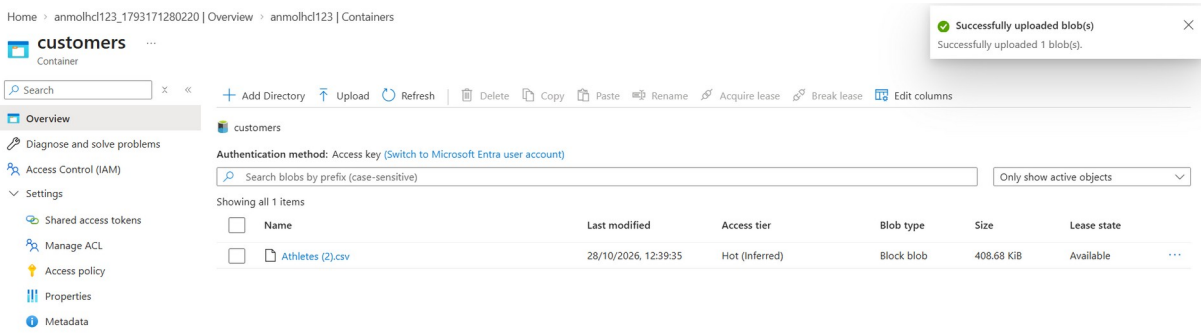
a. Configure a storage account with hierarchical namespace enabled. (1 mark)

Open the Azure Portal and create a Storage Account. Select the appropriate subscription and resource group, then enable the Hierarchical namespace option to use Azure Data Lake Storage Gen2.





b. Create a filesystem or container and upload a sample dataset. (1 mark)



Open the storage account, go to Containers, create a container, and upload a sample CSV dataset for analytics.

c. Explain how Microsoft Entra ID and Azure RBAC manage access to the data. (2 marks)

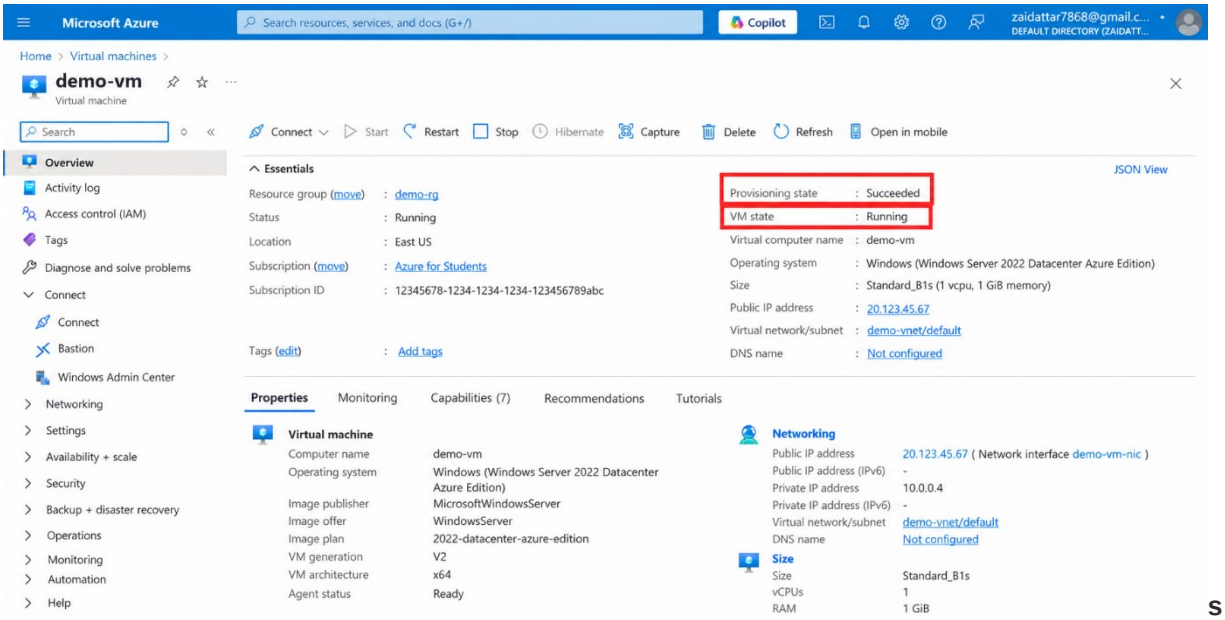
Microsoft Entra ID manages user identities and authentication. Azure RBAC assigns permissions to users or groups based on their roles, such as Storage Blob Data Reader or Storage Blob Data Contributor, to control access to the data.

d. Explain how you would verify that unauthorized users cannot access the dataset. (1 mark)

Sign in as a user without the required permissions and attempt to access or download the dataset. Verify that access is denied, and check the Azure Monitor or diagnostic logs if configured

Question 16: Azure VM Troubleshooting (5 Marks)

a. Verify the VM's provisioning and power states.



S

b. How can NSG rules prevent connectivity?

- Network Security Group (NSG) rules control inbound and outbound traffic.
- If inbound rules block ports such as 22 (SSH) or 3389 (RDP), administrators cannot connect to the VM.

c. How do subnet and routing configurations affect connectivity?

- Incorrect subnet configuration or route table entries can block traffic to the VM.
- Verify the subnet, route table, and network connectivity settings.

d. Recommend a secure troubleshooting method.

- Use Azure Bastion to securely connect to the VM through the Azure Portal without exposing its public IP address.
- Alternatively, use Just-in-Time (JIT) VM access to open required ports only when needed, reducing exposure to the public internet.

Question 17: Azure Data Integration (5 Marks)

A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

a. Select an appropriate service to orchestrate data movement.

Check the VM's status in the Azure portal under Virtual Machines. Verify that the provisioning state is Succeeded and the power state is Running.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 mark)

Network Security Group (NSG) rules may block incoming connections if the required port, such as SSH (22) or RDP (3389), is denied. Verify that the inbound rules allow traffic from authorized sources.

c. Describe how to organize files by ingestion date. (1 mark)

Incorrect subnet configurations, route tables, or user-defined routes may prevent traffic from reaching the VM or returning to the client. Check the subnet association, route table, and network connectivity.

d. Explain how to handle malformed records and missing values. (1 mark) e. Explain how to monitor pipeline execution and validate data completeness. (1 mark)

Use Azure Bastion to connect securely to the VM through the Azure portal without assigning a public IP address to the VM. Alternatively, use Azure Serial Console to troubleshoot supported boot and connectivity issues. Restrict NSG rules to authorized sources and avoid opening ports to the entire internet.